Pixler — Product Specification

Version: 1.0 (Draft) **Last updated:** May 23, 2026 **Owner:** Mike (Lazar team) **Status:** Pre-implementation

---

1. Overview

1.1 What Pixler is

Pixler is a **local-first desktop application** that orchestrates Claude Code (and other agent CLIs) to take software tickets from "Todo" to "Merged PR" end-to-end. Each teammate runs their own copy via `npx pixler`. The app boots a local Node.js server and opens a browser to `localhost:7777`.

Conceptually it sits in the same category as Conductor.build, but is open-source, cross-platform (Mac/Windows/Linux), built on a stack the Lazar team already uses, and adds opinionated features around Linear-driven workflows, plan-file durability, and token-efficient agent communication.

1.2 Distribution and authentication model

- Shipped as an npm package, executed via `npx pixler`
- Pixler itself is free
- All LLM compute is delegated to **already-authenticated CLIs on the user's machine**: `claude`, `codex`, `gemini`, plus `gh` for GitHub
- This keeps cost on the user's existing Pro/Max/Codex subscription
- Pixler never holds an LLM API key and never makes its own LLM API calls in v1/v2

1.3 Tagline

"Local agentic SDLC orchestrator. Your stack, your team, zero LLM API cost."

1.4 Success criteria

Pixler is working when:

1. A teammate takes a Linear ticket from Todo to merged PR without leaving the app

2. Three teammates pulling `docs/plans/ENG-X.md` from main see identical context for ENG-X
3. Running 3 agents in parallel doesn't clobber any files
4. The team forgets Conductor exists

---

2. Stack

Layer	Choice	Rationale
Backend	NestJS (Node)	Matches Lazar muscle memory; clean DI for orchestrator modules
Frontend	React 19 + Vite + TypeScript	Framer Motion + assistant-ui + future React Native path
Styling	Tailwind v4 + CSS variables + `@pixler/tokens`	Runtime theme switching with no rebuild
Animation	Framer Motion	Gold standard for declarative React animation
Gestures	`@use-gesture/react`	Multi-touch, pinch, fling, with Framer Motion integration
Drawers/sheets	Vaul	Finger-tracked dismiss with velocity-and-distance physics
Command palette	cmdk	Vercel's primitive; fuzzy search everything
Primitives	Radix UI	Accessibility + unstyled by default
Chat UI	**assistant-ui**	Production chat primitives (Thread, Message, Composer, ActionBar); streaming, markdown, code blocks, attachments out-of-box; theme via CSS variables
Icons	Lucide React	Matches Lazar
State	Zustand (UI) + TanStack Query (server)	Local UI state + reactive cache

Realtime	Socket.io	Stream PTY output + agent events
Terminal	node-pty + xterm.js	Spawn native shells, render in browser
Local DB	better-sqlite3	Industry standard for local desktop apps
Settings file	JSON at `~/.config/pixler/config.json`	Human-editable, version-controllable per user
Git	simple-git + native `git worktree`	Native git is faster and more reliable than libgit2
Linear (Pixler internal)	`@linear/sdk` (GraphQL)	For sync, status transitions, attachments
Linear (agent-facing)	**Thin Pixler CLI wrapper** (default) + MCP option	CLI is ~98% cheaper in tokens than MCP, user can switch
GitHub	`gh` CLI (shells out, uses existing auth)	Zero schema cost, leverages user's existing auth
Mobile (v3+)	Expo + React Native + Reanimated 3 + NativeWind	Read-only companion
Repo layout	Turborepo + pnpm workspaces, standalone	Mirrors Lazar patterns
Distribution	npm package with `bin` entry	`npx pixler` boots server + opens browser

2.1 Repo structure

```
pixler/
├── apps/
│   ├── api/                    # NestJS backend (the local server)
│   └── web/                    # React + Vite frontend
├── packages/
│   ├── ui/                     # React component library (port of Lazar ui)
│   ├── tokens/                 # Design tokens (themes, spacing, typography)
│   ├── shared-types/           # DTOs, Linear types, agent events
│   ├── orchestrator/           # Pure agent state-machine logic (testable)
│   └── linear-cli/             # Thin Linear CLI agents call via bash
├── bin/
│   └── pixler.js               # npx entry: spawns server + opens browser
```

```
└── package.json                # "bin": { "pixler": "./bin/pixler.js" }
```

2.2 Design tokens approach

Tokens live in `@pixler/tokens` and are consumed by both web (Tailwind + CSS variables) and future mobile (NativeWind). The package is generated from Lazar's existing Tailwind preset as the starting point (the "Forest" theme), with the other 7 themes following the same token shape. Lazar can also consume `@pixler/tokens` so updates propagate to both products.

---

3. Core concepts

3.1 Workspace

A workspace is the primary unit. One workspace = one ticket (or chat session) + one git worktree + one branch + one agent. Workspaces appear in the left sidebar with a status badge.

3.2 Plan file

Markdown file at `/docs/plans/<TICKET-ID>.md` in the repo. Created by the planner agent, lives on the branch, gets committed and merged. The plan file ties Linear ↔ GitHub together and lets the workflow survive across machines. Section 5 covers storage modes in detail.

3.3 Worktree

Each workspace gets an isolated git worktree at `<repo>/../pixler-worktrees/<workspace-name>/`. Branch naming defaults to `pixler/<workspace-name>` (configurable per project).

3.4 Setup script

Bash script that runs once when a workspace is created. Copies `.env`, installs deps, seeds databases, generates per-workspace identifiers. Configured per project, committed to the repo in `pixler.json` so teammates share it.

3.5 Run script

Optional script that launches the workspace's dev environment (`pnpm dev`, `pnpm api:dev`, etc.). Uses `$PIXLER_PORT` so multiple workspaces run side-by-side without port collisions.

3.6 Pixler environment variables

Available in setup, run, and archive scripts:

Variable	Description
`$PIXLER_ROOT_PATH`	Original repo root
`$PIXLER_WORKSPACE_PATH`	This workspace's worktree path
`$PIXLER_WORKSPACE_NAME`	This workspace's name (used for per-workspace resources)
`$PIXLER_PORT`	Auto-assigned port for this workspace
`$PIXLER_TICKET_ID`	Linear/GitHub issue ID (if applicable)
`$PIXLER_BRANCH`	Branch name

---

4. The agent loop

4.1 State machine

Every workspace flows through this state machine:

```
IDLE
 ↓
PLANNING ──→ REVIEWING ──┬──→ AWAITING_APPROVAL ──→ EXECUTING
                         │      (human gate)
                         └──→ (rejected, loop back, max 3×)

EXECUTING ──→ VALIDATING ──→ PR_OPEN ──→ DONE
```

4.2 Phase commands

Phase	Default command (configurable per project + per role)
Planning	`claude --permission-mode plan "Work ticket [details]. Write plan to docs/plans/<id>.md"`
Reviewing	`codex exec "Review this plan: [contents]. Reply REJECTED with critique or APPROVED."`
Executing	`claude --execute "Implement plan at docs/plans/<id>.md"`

Validating	`claude --execute "/review"`
PR	`gh pr create --fill --body-file <generated-body>`

All commands run inside the worktree's cwd. All output streams to the chat/terminal UI via WebSocket.

4.3 Approval gates

Three gates by default, each independently toggleable to auto-approve:

1. **Plan approval** — after planning + review converge
2. **Validation approval** — after `/review` runs post-execution
3. **PR approval** — before opening the PR

Per-project setting toggles auto-approve for any combination of the three.

4.4 Loop limits

Plan → review loops are capped at 3 iterations before human intervention. Configurable per project (1–5).

4.5 Mode: Chat vs Terminal

Both modes use the same underlying `claude` (or `codex`) interactive CLI spawned via node-pty. They differ only in UI presentation. Both bill against the user's subscription.

Mode	UI	Use when
Chat *(default)*	assistant-ui polished chat panel, styled like claude.ai	You want a conversational interface, message history, markdown rendering, attachments
Terminal	Raw xterm.js Big Terminal view of the `claude` CLI	You want full control over Claude Code's slash commands, status line, raw output

The mode picker appears in the New Workspace dialog. Users can switch modes mid-workspace (the underlying process is the same).

A future v3 **Headless** mode will use `claude -p` (non-interactive Agent SDK), which on/after June 15, 2026 bills against a separate Agent SDK credit pool ($20/$100/$200 monthly depending on plan, metered at API rates after credit is exhausted). The mode picker shows a clear warning when Headless is selected.

---

5. Plan storage

5.1 Three storage modes

Mode	Where the plan lives	Best for
File	`/docs/plans/<TICKET>.md` committed to branch	Complex features, multi-step work
Inline	Appended to Linear ticket description, delimited by HTML comment markers	Simple tickets, rename/cleanup tasks
Attachment	Uploaded to Linear via `fileUpload` mutation; rolling pair (current + previous)	Medium tickets, want plan visible in Linear without polluting repo

5.2 Default behavior

Global default storage method: **Auto** — Pixler picks per ticket complexity.

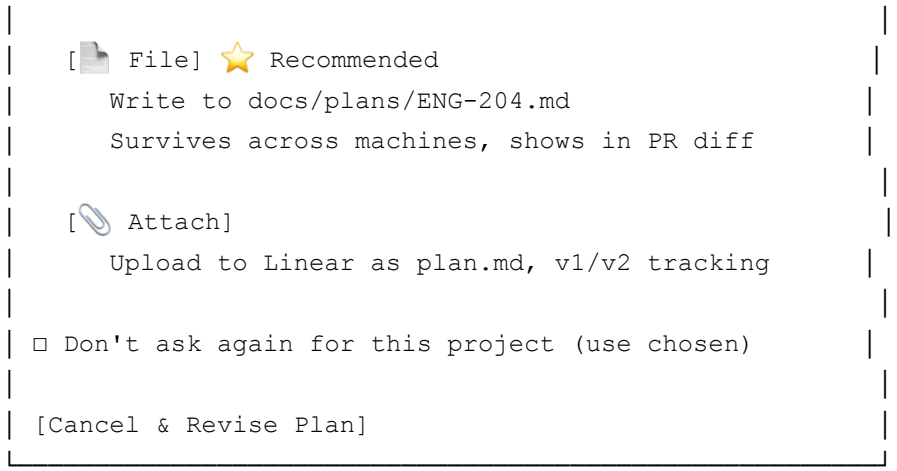
Auto-recommendation signals:

Signal	Suggested mode
Description < 200 chars, no acceptance criteria, no sub-tasks	Inline
Description has acceptance criteria OR estimate ≥ 3 OR has sub-tasks	File
Description medium-length, no sub-tasks, labeled “bug” or “chore”	Attachment
Has a `pixler:plan-file` / `pixler:plan-inline` label	Forced
Project setting overrides default	Project wins

5.3 Big-plan prompt

When a plan starts as **inline** but exceeds thresholds (default: > 3 tasks OR > 500 chars approach), Pixler **blocks** and forces a choice:

```
┌─ Plan exceeds inline thresholds ──────────────────────┐
│ This plan has 7 tasks and 1,240 chars.                │
│ Inline storage is configured for ≤3 tasks.            │
│                                                       │
│ Choose storage for this plan:                         │
```



- No default action, no Enter-to-confirm
- File is pre-recommended with a star badge
- "Don't ask again" writes the chosen mode to `pixler.json` as the project default
- "Cancel & Revise Plan" loops back to the planner with a "make it smaller" instruction
- Project Settings → Plans → **"Reset prompts"** clears all "don't ask again" flags

5.4 Attachment versioning (rolling pair)

Plan revision	Linear attachments after this revision is committed
v1	`plan-v1.md` (only)
v2	`plan-v2.md` (current) + `plan-v1.md` (previous)
v3	`plan-v3.md` (current) + `plan-v2.md` (previous) — `plan-v1.md` deleted
v4	`plan-v4.md` (current) + `plan-v3.md` (previous) — `plan-v2.md` deleted

Always max two attachments past v1. Old ones deleted via Linear's `attachmentDelete` mutation immediately after the new one uploads.

Frontmatter tracks both:

```
---
ticket: ENG-204
storage: attachment
revision: 3
linear_attachment_current: att_xyz789
linear_attachment_previous: att_abc123
```

```
---
```

5.5 Plan file template

```
---
ticket: ENG-101
storage: file
linear_url: https://linear.app/lazar/issue/ENG-101
branch: pixler/eng-101-fix-auth
workspace: crimson
created_by: pixler
created_at: 2026-05-23T14:00:00Z
status: in_progress
revision: 1
planner: claude-sonnet-4-7
reviewer: codex
---

# ENG-101: Fix Auth Redirect

## Acceptance Criteria
[Copied from Linear]

## Approach
[Planner's prose]

## Tasks
- [ ] Analyze /auth/callback endpoint
- [ ] Inject JWT into cookie with proper SameSite
- [ ] Add E2E test
- [ ] Update docs

## Peer Review
**Verdict:** APPROVED (round 1)
[Reviewer's full critique]

## Execution Log
- 14:02 Created: src/auth/callback.controller.ts
- 14:05 Modified: src/auth/auth.service.ts
- 14:09 Tests passing
- 14:11 PR opened: #428
```

5.6 Inline mode markers

For inline storage, Pixler appends to the Linear ticket description below the user's content,

wrapped in HTML comments so Pixler can find and update its section without touching anything the human wrote above:

```
[original ticket description]

---
<!-- pixler-plan:start revision=2 -->
## Plan (Pixler)

**Approach:** [...]

- [x] Replace all call sites
- [x] Update tests
- [ ] Update docs
<!-- pixler-plan:end -->
```

5.7 Reset behavior

- **Project Settings → Plans → "Reset plan prompts"** clears that project's "don't ask again" flags
- **Project Settings → General → "Reset to defaults"** resets all repo settings to global defaults
- **Global Settings → Storage → "Reset all prompts"** clears every "don't ask again" across every project

---

6. Linear integration

6.1 Source of truth

Linear is the source of truth for tickets. GitHub hosts code and PRs only.

6.2 Auth

PAT auth in v1; OAuth in v2.

6.3 Sync behavior

- Sync assigned Todo + In Progress tickets every 60s and on-demand
- Create workspace directly from Linear ticket — title, description, acceptance criteria, labels passed to planner

6.4 State transitions

Event	Linear state
Workspace start	Todo → In Progress
PR open	In Progress → In Review
PR merge	In Review → Done

State name mapping is configurable per global and per project (your team may use different names).

6.5 Sub-tasks

- Plan checkboxes become Linear sub-issues on plan commit
- Ticked checkboxes during execution close the corresponding sub-issues in real time

6.6 Pixler deep links in Linear

When a workspace is created, Pixler posts a Linear comment:

```
🎼 Open in Pixler → pixler://workspace/ENG-101
   Plan: docs/plans/ENG-101.md (file)
```

Clicking the link triggers OS URL-scheme handling, opens (or focuses) Pixler, and navigates directly to that workspace.

6.7 CLI vs MCP for agents

Mode	Tokens per op	Reliability	Recommended
Pixler Linear CLI (default)	~80 tokens (–help text)	100%	✓
Linear MCP server	~5,000–15,000 tokens per turn (schema overhead)	~72% on complex tasks	Power users only
Both	CLI for cheap ops, MCP for richer interactions	—	—

Default ships with Pixler Linear CLI. Project Settings → Integrations lets the user switch to MCP or enable both.

Pixler itself always uses `@linear/sdk` internally for sync, status transitions, attachments — never goes through an agent.

6.8 Pixler Linear CLI commands

The `@pixler/linear-cli` package exposes thin commands agents call via bash:

- `pixler ticket fetch <id>` — full ticket details + comments + sub-issues
- `pixler ticket comment <id> "..."` — add comment
- `pixler ticket state <id> <state>` — transition
- `pixler ticket label <id> <label>` — add/remove label
- `pixler subissue create <parent-id> "title"` — create sub-issue
- `pixler subissue complete <id>` — close sub-issue
- `pixler attachment upload <ticket-id> <file>` — upload, returns `att_id`
- `pixler attachment delete <att-id>` — delete

Each command's `--help` is short and tight (~80 tokens), so the agent learns the surface on first use and never re-reads.

---

7. GitHub integration

7.1 Auth

Uses `gh` CLI from the user's terminal environment. Pixler verifies via `gh auth status` and surfaces a re-auth helper if missing.

7.2 Repo onboarding

Two paths in the New Project flow:

- **Open local folder** — point at any existing git repo
- **Clone from GitHub** — paste URL, Pixler clones to `~/pixler/repos/<name>/`

7.3 PR lifecycle

- PR creation via `gh pr create` with auto-generated body referencing plan file + execution log

- PR checks and CI status surfaced in the workspace's Checks tab
- Inline review comments visible in UI
- Merge from Pixler when checks pass + approvals in place

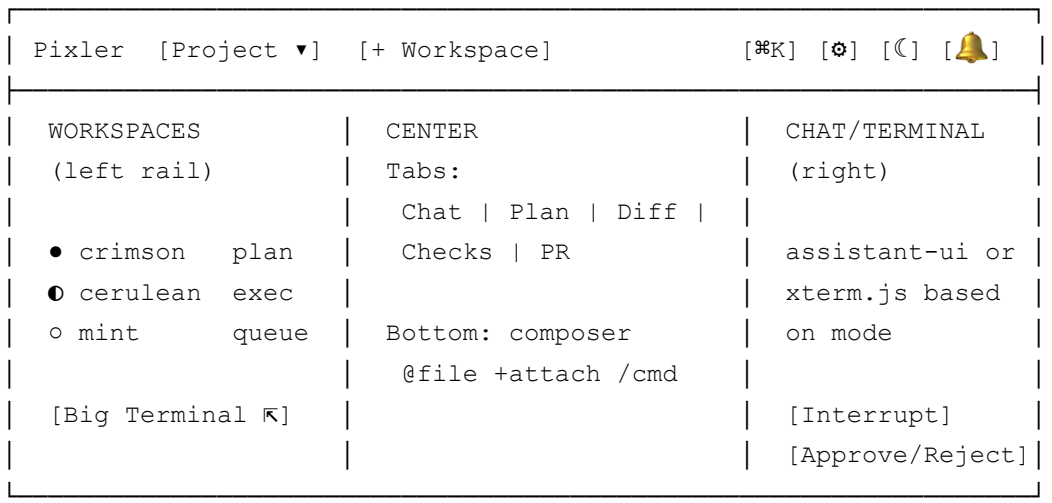
7.4 CI status (v1 vs v2)

- **v1:** poll `gh pr checks <pr>` every 30s while a PR is open; show static badge
- **v2:** real-time push via GitHub webhooks (requires Pixler to be reachable; uses tunneling for local-first install)

---

8. UI layout

8.1 Three-pane default



All panes resizable. Any pane can go full-bleed. Big Terminal mode goes full-bleed for the chat/terminal pane.

8.2 Design philosophy

- **Icon-first.** No labels when shape is obvious. Tooltips on long hover for discoverability.
- **3-dot overflow** on every panel header for secondary actions, keeping primary surfaces clean.
- **Settings opens as a Vaul side drawer** (~480px from right) — not a modal, not a full page. You can settings-while-doing.

- **Toggles over checkboxes**, segmented controls over radios, steppers over number inputs.
- **All transitions Framer-Motion'd** so the UI feels kinetic, not static.
- **Theme picker is a grid of 16 swatches** (8 themes × light/dark) with live hover preview.

8.3 Workspace naming

Source	Default name
From Linear/GitHub issue	`<label-short>-<id>-<3-word-slug>` (e.g., `bug-eng-101-fix-auth-redirect`)
If derived name collides with open workspace	Fall back to color name
From scratch (no ticket)	Color name

Color name pool (~50 names, alphabetical, picked round-robin from unused): `amber`, `cerulean`, `coral`, `crimson`, `indigo`, `lavender`, `mint`, `ochre`, `slate`, `teal`, …

Each color name subtly tints the workspace's accent in the sidebar.

Users can rename freely. Branch names follow the workspace name (`pixler/<workspace-name>`) unless overridden in project settings.

8.4 Gesture and touch UX

- **Swipe-to-archive** on workspace cards (Gmail-style; > 50% or fast fling)
- **Pull-to-refresh** on Linear sync
- **Vaul drawers** for settings, theme picker, command palette
- **Pinch-to-zoom** diff viewer
- **Long-press** workspace card for context menu
- All gesture patterns translate to React Native for mobile companion via react-native-gesture-handler + Reanimated 3

8.5 Chat UI specifics

- Built on **assistant-ui** primitives (Thread, Message, Composer, ActionBar, ThreadList)
- Streaming, auto-scroll, markdown rendering, code syntax highlighting via Shiki

- @file and @folder tagging from composer
- Attach: screenshots, logs, specs, PDFs, .md files
- Slash commands: `/plan`, `/review`, `/test`, `/commit`, `/rebase`, `/resolve-conflicts`
- Thinking mode + plan mode toggles
- Stop/interrupt running agent
- Message history per workspace, searchable
- **Theming via CSS variables** — chat UI adopts whichever Pixler theme is active, no separate config needed
- Unread badges + jump-to-next-unread (Conductor pattern)
- Image click-to-fullscreen in a file tab

8.6 Terminal UI specifics

- xterm.js streaming live node-pty output
- Multiplexed across workspaces (dropdown to switch)
- **Shadow mode:** take over the agent's session at any time ("Interrupt")
- Respects user's tool version manager (mise, asdf, nvm, rbenv) automatically
- Terminal theme syncs with Pixler theme
- User's existing custom Claude Code status line continues to work; Pixler optionally adds a Pixler-specific status row above/below (v2)

8.7 Diff viewer

- Monaco-based with syntax highlighting per repo language
- Full file diff + inline diff modes
- Per-file filter, search within diff
- Stage/unstage hunks before commit
- Side-by-side or unified view (toggle)
- Updates live as the agent writes files
- Keyboard shortcut: `⌘+Shift+D`

8.8 Checks tab

Consolidates per workspace:

- Git status (uncommitted, staged, ahead/behind)
- CI runs from GitHub (polled v1, webhook v2)
- Deployment previews (Vercel/Netlify if configured)
- PR review comments
- Todos extracted from the plan file
- Tests run from terminal, Run button, or spotlight testing (v2)

8.9 Command palette

- `⌘K` / `Ctrl+K`
- Fuzzy search every action, setting, workspace, ticket, file
- Recent actions surface first
- Keyboard navigation throughout
- Every setting reachable as a searchable action (e.g., "toggle auto-approve plan")

8.10 Activity feed

Two surfaces:

- **Toast stack** — top-right, persistent across modes, respects do-not-disturb
- **Activity tab** in the left rail — full history, filterable per workspace
- Available regardless of which mode (Chat/Terminal/Diff/Checks) the user is in
- Events: plan ready, approval needed, agent stuck/blocked, PR opened, CI failed/passed, PR merged, workspace archived

8.11 Run/Open App

- **Run button** in workspace toolbar — starts the project's run script
- Status: running / stopped / crashed; click to view live logs in a dedicated tab
- **"Open app" button** appears once run script signals ready, opens `http://localhost:$PIXLER_PORT` in default browser

- `$PIXLER_PORT` auto-assigned per workspace so parallel workspaces don't collide
- **Stop button** kills the process cleanly

8.12 External IDE launcher

- Auto-detects installed IDEs on first launch: VS Code, Cursor, Windsurf, WebStorm, Zed, Sublime Text, Vim/Neovim, Xcode
- "Open in…" dropdown on every workspace card
- Default IDE setting (Settings → External Tools)
- Keyboard shortcut: `⌘+E` opens current workspace in default IDE at the right worktree path
- Custom IDE option: user provides command path + args template

---

9. Theming

9.1 8 themes × light/dark = 16 variants

Theme	Description
Forest *(default; sourced from Lazar tokens)*	Green-primary palette from Lazar's design system
Graphite	Neutral grays, blue accent — Linear-ish
Catppuccin	Frappé (dark) / Latte (light) — matches existing terminal aesthetic
Tokyo Night	Deep blues + magenta accent
Nord	Cool blue-gray, minimal
Rosé Pine	Warm muted, beloved in dev circles
Solarized	Classic high-readability palette
Mono	Pure grayscale, accent-free — for minimalists

9.2 Implementation

All themes driven by CSS custom properties. Theme switch = single attribute flip on `<html>`, zero re-render, no flash. Tokens read via Tailwind utility classes (`bg-base`, `text-primary`, etc.). Tokens live in `@pixler/tokens` for cross-product sharing with Lazar and future mobile.

9.3 Mode selection

- **Light / Dark / System** (System follows OS preference, default)
- Per-project theme override available (e.g., Lazar uses Forest, side projects use Mono)

9.4 Terminal and chat sync

Both the xterm.js terminal and the assistant-ui chat panel read from the same CSS variables. Picking a theme changes everything coherently.

---

10. Settings architecture

10.1 Three-tier cascade

- **Global** (`~/.config/pixler/config.json` + SQLite) — applies app-wide
- **Per-project** (`pixler.json` committed to repo + SQLite overrides) — overrides global per key
- **Per-workspace** (SQLite row, keyed by workspace ID) — overrides project per key for that workspace only

10.2 Global Settings categories

Opened via the gear icon in the top bar. Icon-rail left, content right.

Category	Contains
Account	Sign-in (future), privacy controls, telemetry opt-in (default on with prominent disclosure), crash reporting
Models	Which Claude Code models appear in the picker, which Codex models, default model per role (planner/reviewer/executor), API key/env var management
Providers	Path to `claude` binary, path to `codex`, path to `gemini`, path to `gh`. Auto-detect button. Re-auth shortcuts (helps user authenticate if missing).

Env	Environment variables passed to every agent process. Key-value editor with secret masking.
Linear	PAT, default workspace, default team, default project, sync interval, **status name mapping** (which Linear states = "Todo" / "In Progress" / "In Review" / "Done")
Git	Default branch naming template, commit message template, PR template, auto-merge behavior, automerge-on-green toggle, default base branch
Plans	**Default storage method** (file/inline/attachment/**auto** — default), file directory, inline thresholds, attachment versioning
Appearance	Theme + mode (light/dark/system), density (compact/comfortable/spacious), font (UI + monospace), terminal theme sync, sidebar width default, animation level (full/reduced/off — respects OS prefer-reduced-motion)
Keyboard	Full shortcut editor with conflict detection, presets (Default / Vim / Emacs-ish)
Notifications	System notifications on/off per event, sound, do-not-disturb hours
Terminal	Default shell, font, font size, cursor style, scrollback lines, copy-on-select, paste warning, tool version manager detection
External Tools	Default IDE, detected IDEs list, custom IDE config, `pixler://` URL scheme registration status
Storage	Worktree base directory, plan cache directory, log retention, **disk usage breakdown** (see 10.5), "Reset all prompts", **"Reset all settings"** (type `RESET`), **"Wipe database"** (type `WIPE EVERYTHING`)
Experimental	Big Terminal toggles, gesture sensitivity, beta features
About	Version, check for updates, changelog link, license, "Submit feedback"

10.3 Project (Repository) Settings categories

Opened via the gear icon next to the project name in the project switcher.

Category	Contains
General	Display name, **icon** (auto-detects `icon.png` / `logo.svg` / `favicon.ico` in repo root, override with `.ico` / `.png` upload), description, **"Reset to defaults"** button

Preferences	Free-form instructions appended to every agent's system prompt for this repo (Pixler-only, doesn't replace `AGENTS.md` / `CLAUDE.md`)
Scripts	Setup script editor (syntax highlighted), run script editor, archive script. **Variables reference panel** showing all `$PIXLER_*` env vars with click-to-copy.
Files to copy	Declarative list of gitignored files/globs to copy into each workspace (`.env`, `.env.local`, `serviceAccount.json`, etc.)
Integrations	Linear: CLI / MCP / Both (default CLI). GitHub: confirm `gh` auth status. Per-project Linear team/project filter, status mapping override, label filters.
Git	Branch template override, base branch, PR template override, auto-merge toggle, merge strategy (squash/merge/rebase)
Plans	Storage method override, plan directory, inline thresholds, **"Reset prompts"** button (clears "don't ask again" flags for this project)
Models	Override which models are used for planner/reviewer/executor in this repo
MCP *(v3)*	MCP servers available to agents in this repo
Workspaces	Max parallel agents, worktree directory override, **remove-confirmation silence window** preference
Theme override	Per-project theme + mode override
Danger	**Remove project from Pixler** (modal confirm — keeps repo on disk), **Delete project** (type project name to confirm — also deletes worktrees + cloned repo if Pixler cloned it)

10.4 Workspace-level controls

Right-click a workspace card or open via 3-dot menu:

- Rename
- Pin / unpin
- Change agent (Claude → Codex mid-stream)
- Change model
- Toggle thinking / plan / fast mode

- Archive
- **Remove** — confirmation modal with **"Silence remove confirmations for 1 minute"** checkbox (resets on app close)
- Open in IDE
- Open app
- Run setup script (re-run)

These are session-level controls and don't persist back to project or global settings.

10.5 Storage / cleanup view

```
Storage
├── Active workspaces (3)              412 MB
│   ├── crimson (ENG-101)              180 MB  [Open ↗]
│   ├── cerulean (ENG-99)              140 MB  [Open ↗]
│   └── mint (ENG-102)                  92 MB  [Open ↗]
├── Archived workspaces (14)           1.8 GB  [Clean up...]
├── Plan attachments cache              48 MB  [Clear]
├── Agent transcripts                   12 MB  [Clear]
├── Setup script logs                    4 MB  [Clear]
└── App data + database                  6 MB
                                      ─────────
                                Total:  2.3 GB
```

Each row shows path on hover, has a copy-path button, click opens in Finder/Explorer.

10.6 Reset confirmation tiers

Action	Confirmation
Reset prompts (don't-ask-again flags)	Single click
Remove workspace	Modal with **"Silence for 1 minute"** checkbox
Remove project (keeps repo)	Modal with Confirm button
Delete project (also deletes worktrees + clone)	Type project name to confirm
Reset all global settings	Type `RESET` to confirm

Wipe database (factory reset)	Type `WIPE EVERYTHING` to confirm

10.7 Team config sharing

Two files committed to the repo:

```
your-repo/
├── pixler.json              # canonical team workflow config
└── .pixler/
    ├── instructions.md      # (v2) Pixler-only repo-wide agent instructions
    └── README.md            # explains what's in .pixler for teammates
```

`pixler.json` covers: setup/run scripts, files to copy, plan directory, branch template, models per role, Linear team/project, plan storage defaults.

When a teammate clones a repo with `pixler.json`, Pixler shows a **diff view** comparing team config to their current settings. They cherry-pick what to import — no aggressive auto-apply.

Personal settings (theme, keyboard shortcuts, notification prefs) stay global and never get touched by repo config — only **workflow** settings come from `pixler.json`.

10.8 Agent instructions inheritance

Pixler reads existing agent-instruction files at workspace creation, in priority order:

1. `AGENTS.md` (Codex-native, increasingly cross-agent standard) — used here, matches Mike's current convention
2. `CLAUDE.md` (Claude Code's native) — fallback if `AGENTS.md` missing
3. `.pixler/instructions.md` *(v2 feature)* — Pixler-only layer for orchestration-specific instructions that shouldn't shape Claude's general behavior

All applicable files are concatenated and passed to every agent in the workspace as a system-prompt prefix.

10.9 Published JSON schema *(v2 / beta)*

When Pixler exits beta and has a public domain, ship a JSON schema at a stable URL (e.g., `https://pixler.dev/schemas/config.json`) so editors autocomplete and validate `pixler.json`. Schema-aware editors (VS Code, JetBrains) pick it up automatically.

---

11. Onboarding

11.1 First-launch flow

Designed to take under 90 seconds. Each step is a Vaul drawer panel sliding in from the right with Framer Motion. Step indicator at top. Every step has a "Skip — I'll do this later" link.

Step 1 — Welcome + Appearance

- Pixler logo, one-sentence pitch
- Theme picker: 8 swatches with hover preview
- Mode: Light / Dark / **System** *(default)*
- "Next →"

Step 2 — Connect Tools

- **Git** — checks `git config user.name` and `user.email`; if missing, walks user through setting them
- **Claude Code** — detects `claude` on PATH; shows version if found ("Claude Code 2.1.59 detected"); if missing, shows install command + copy button + "Re-check" button; detects subscription type (Pro/Max/API key) and surfaces which is active
- **gh CLI** — checks `gh auth status`; if missing, walks user through `gh auth login`
- **(Optional)** Codex / Gemini for peer review with the same auto-detection pattern
- Big "Re-check all" button
- "Next →"

Step 3 — Connect Linear

- "Paste your Linear PAT" (link to Linear's API token page)
- PAT hidden after paste, validated immediately via `viewer` query
- Shows: "Connected as [name] in [workspace]"
- Pick default team from dropdown (auto-fills if user only has one)
- "Skip if you'll use GitHub Issues" link

- “Next →”

Step 4 — Add First Project

- Three big tiles: **Open local folder** / **Clone from GitHub** / **Skip for now**
- Local: file picker → detects git remote, default branch, package manager
- GitHub: paste URL → clone with progress
- Once added: tiny “Project Settings” preview shows key defaults (plan storage = auto, branch template = `pixler/<workspace>`)
- “Finish — let’s go →”

Step 5 — Telemetry consent

- Single toggle: **“Help improve Pixler by sharing anonymous usage data”** — checked by default
- Expandable “What gets sent?” showing exact fields (feature usage counts, error rates, model selection patterns — **no code, no prompts, no ticket content**)
- “Finish”

11.2 Post-onboarding nudge

User lands on the empty workspaces view with a single CTA: **”+ Create your first workspace”** — clicking opens a guided variant of the New Workspace dialog with inline hints that fade after first use.

11.3 Re-runnable

Help menu → **“Re-run onboarding”** — useful when adding a new teammate’s machine.

---

12. Token efficiency

12.1 Why it matters

The 5-hour rolling rate-limit window means burning tokens cheaply matters more than ever. Pixler defaults aim to minimize token cost for the same work.

12.2 Built-in token-saving defaults

- **CLI > MCP** for Linear and similar integrations (~98% reduction)
- **`.claudeignore`** seeded with `node_modules/`, `dist/`, `.next/`, `*.lock`, build artifacts on first workspace creation
- **Auto `/clear`** when switching workspaces
- **Auto `/compact`** suggestion when session approaches context limit
- **Plan mode default on** for tickets above complexity threshold (prevents costly rework)
- **Per-project MCP allowlist** *(v3)* — only load MCP schemas the project actually uses
- **Haiku for refactor/doc/cleanup tasks** suggestion when Pixler detects a simple change is needed; Sonnet for planning; Opus when explicitly requested

12.3 Token health panel

Settings → Usage shows:

- Current 5-hour window usage (% used + time to reset)
- Per-model usage breakdown
- Per-workspace token cost
- MCP schema overhead (which MCPs you have loaded, what they cost per turn)
- One-click "disable MCP for this workspace" toggle
- Daily/weekly/monthly historical from local JSONL parsing (~/.claude/projects/)

12.4 Status bar (always visible)

Top-right pill: `73% / 4h 12m` — current 5-hour window + time to reset. Color shifts green → yellow → red as you approach the limit. Click to expand into a popover with model breakdown.

12.5 Pre-flight check before spawning parallel agents

> You're at 78% of your 5-hour window. Starting 3 parallel agents may exhaust it in ~25 minutes. Proceed?

---

13. Checkpoints

13.1 What they are

Auto-snapshots taken at key points so users can roll back if an agent goes off the rails.

13.2 When taken

- Before plan execution starts
- Before each major file write batch (heuristic: > 5 files or > 200 lines)
- Before `/resolve-conflicts`
- Before `/rebase`
- Manually via "Take checkpoint" button or `⌘+K` → "Checkpoint"

13.3 How they work

Each checkpoint is a labeled `git stash push` of working-tree state + a snapshot of the workspace state (chat history, plan revision, todo state) stored in SQLite. Rollback re-applies the stash and restores workspace state.

13.4 UI

- Checkpoints tab in the workspace center pane
- List of checkpoints with timestamp, label, file count, line count
- One-click rollback (with confirmation if changes since checkpoint would be lost)

---

14. Build phases

Phase	Duration	Deliverable
1. Shell	2 weeks	`npx pixler` boots, opens browser, streams a live PTY into xterm.js. Theme system with 2 themes wired (Forest from Lazar + Graphite). Vaul settings drawer skeleton. ⌘K stub.
2. Projects + Linear	2 weeks	Add project (local + GitHub clone). Linear PAT auth, ticket list, state transitions. Project Settings UI. `pixler.json` read/write. Pixler Linear CLI wrapper.
3. Orchestrator + worktrees	2 weeks	Workspace creation, setup scripts, run scripts, plan → review → execute → PR loop. Plan file format with all 3 storage modes. End-to-end ticket-to-PR demo. Checkpoints.
4. Chat UI +		assistant-ui chat panel, Monaco diff viewer, Checks tab, slash

diff viewer	2 weeks	commands, activity feed, Run/Open App button, IDE launcher.
5. Polish + theming	2 weeks	All 8 themes, gesture polish, keyboard shortcuts, archive/restore, command palette completion, deep links, token health panel.
6. Internal release	1 week	Ship to the team. Telemetry, crash reporting, README, onboarding video.

Estimated total to internal release: ~11 weeks.

---

15. Versioning roadmap

v1 (initial release)

Workspaces, plan loop (file/inline/attachment), Linear+GitHub via CLI, Chat mode (assistant-ui), Terminal mode, Monaco diff viewer, run/open app, deep links, IDE launcher, all 8 themes, full settings UI, onboarding, checkpoints+rollback, CI status via polling, activity feed, token health panel, telemetry opt-out.

v2

Headless mode (Agent SDK with credit-pool billing), `.pixler/instructions.md` Pixler-only instruction layer, Pixler status bar above terminal with token usage, real-time CI via GitHub webhooks, HTML inline preview for static content, agent personalities, published JSON schema for `pixler.json`, Linear OAuth (replacing PAT-only).

v3

Multi-repo `/add-dir`, MCP server config per repo, multi-agent collaboration (experimental), Spotlight testing, submit-a-prompt feedback channel, **mobile read-only companion** (Expo + Reanimated), enterprise managed-settings file (`~/.config/pixler/managed.json` with schema-locked controls).

---

16. Out of scope

Intentionally not in Pixler's scope, at any version:

- Hosted multi-tenant version

- Built-in LLM API calls (always shells out to authenticated CLIs)
- Built-in code editor (Pixler links out to user's IDE)
- IDE plugins (VS Code extension, JetBrains plugin)
- Non-git VCS support
- Replacing Claude Code's chat — Pixler orchestrates, Claude Code converses

---

17. Open questions

These need decisions before / during v1 implementation:

1. **Lazar Tailwind tokens import** — Mike to share preset files; will be extracted into `@pixler/tokens` as the Forest theme anchor
2. **Branding / logo direction** — pending
3. **License** — MIT or Apache 2.0 for open-source vs closed-source-but-free (Conductor's model)
4. **Domain for published JSON schema** — pending (used only in v2)
5. **Color name pool finalization** — currently ~50 alphabetical color names; needs concrete list

---

18. Glossary

Term	Meaning
Workspace	One ticket + one worktree + one branch + one agent. Pixler's primary unit.
Plan file	`/docs/plans/<TICKET>.md` (or Linear inline/attachment). Source of truth for what the agent is doing.
Worktree	Isolated git working directory at `<repo>/../pixler-worktrees/<name>/`.
Setup script	Bash script that prepares a fresh workspace (deps, .env, etc.)
Run script	Bash script that starts the workspace's dev environment.

Checkpoint	Auto-snapshot of working tree + workspace state for rollback.
Big Terminal	Full-bleed terminal mode (raw `claude` CLI).
Chat mode	Polished assistant-ui chat panel over the same `claude` CLI.
Headless mode *(v3)*	Non-interactive `claude -p` runs; bills against Agent SDK credit pool.
Peer gate	The plan review loop where a second LLM reviews the planner's output.
`pixler.json`	Repo-committed team workflow config.
`@pixler/tokens`	Shared design token package (themes, spacing, typography).
Color name	Workspace identifier when no ticket-derived name exists or there's a collision.